Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters

Allocating inference compute under a fixed budget

Zhenbang Lyu

Heidelberg University
Institute of Computer Science

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar
2024, arXiv:2408.03314v1

Can Language Models Benefit from More Thinking?

Humans often think for longer about difficult problems to reach better answers and improve their decisions.

Extra thought gives us time to reconsider an initial answer and work through the reasoning more carefully.

Can language models also improve their answers by using more computation at test time?

The paper studies how a trained model can use an additional inference budget on a challenging question.

How Do We Work through a Math Problem?

Suppose we want a better chance of finding the correct answer to a math problem.

- **Ask several people to solve it independently.** Collect their complete solutions, then compare the answers.
- **Build on an existing attempt.** Read someone else's working and use it to develop an improved solution.
- **Ask for feedback halfway through.** Show someone our unfinished reasoning before deciding how to continue.

A model can also try independently, revise earlier work, or use feedback during generation.

Two Directions for Using Extra Computation

Baseline Generate N independent solutions, then select an answer with a verifier: **best-of-N**.

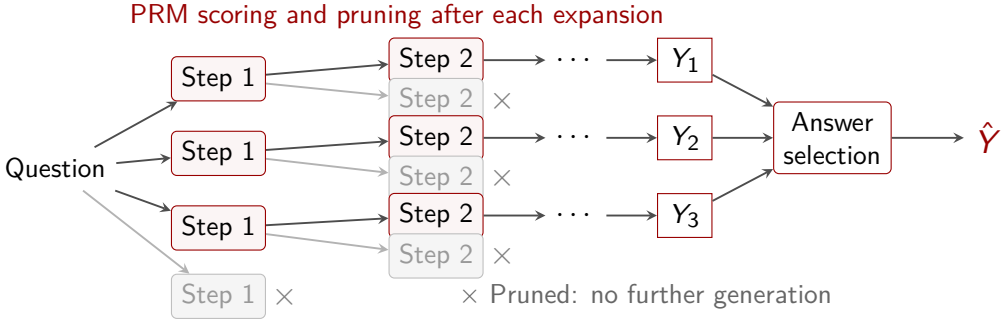
The paper improves on this baseline in two directions.

- **Change what the model generates.** Give it earlier answers and ask for a revised solution. **Revisions** change its **proposal distribution**.
- **Change how feedback guides generation.** Score partial solutions and choose which paths to develop. This is **verifier-guided search**.

We first follow the verifier-guided search route from intermediate steps to a final answer.

PRM-Guided Search from Steps to an Answer

Score partial solutions and stop low-scoring branches before they become complete answers.

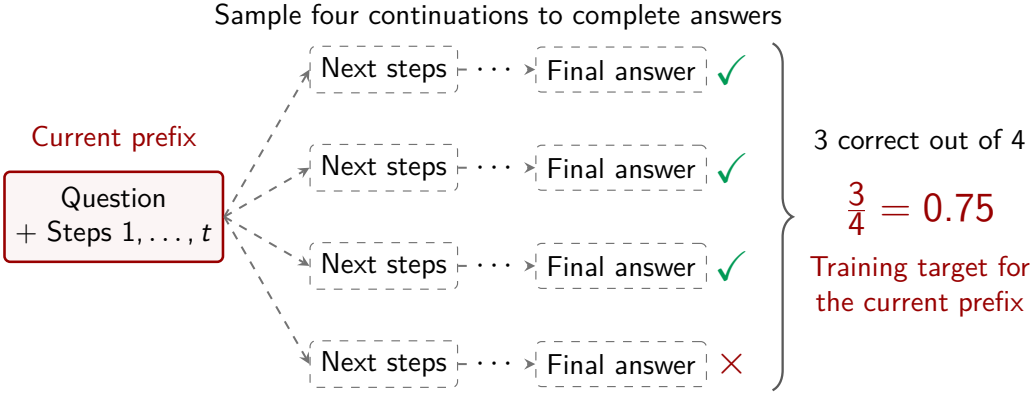


Repeat until completion or the expansion limit. Three decisions remain:

1. Who scores the reasoning?
2. How do we select the final answer?
3. Which candidates should the search retain and expand?

Who Can Score the Reasoning?

Train a **process reward model (PRM)** to predict the chance of finishing correctly from a given prefix.



Illustrative training rollouts. Dashed paths are possible futures.

During search, the trained PRM **scores new prefixes directly**.

How Do We Select the Final Answer?

Use the PRM's **last-step score** for each complete solution, then sum scores for matching final answers.

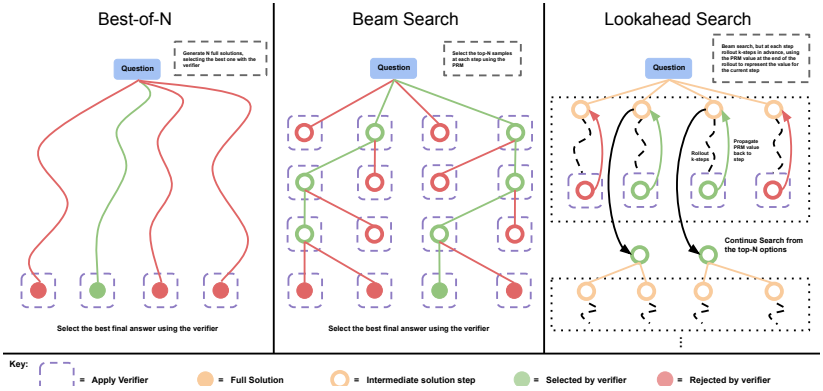
Example candidate	Final answer	Last-step score
Y_1	$x = 2$	0.60
Y_2	$x = 2$	0.55
Y_3	$x = 3$	0.90

$$\text{score}(x = 2) = 1.15, \qquad \text{score}(x = 3) = 0.90$$

Weighted best-of-N selects $x = 2$.

Standard best-of-N would pick the single highest-scoring candidate. Majority voting counts answers without weights.

Which Candidates Should We Develop?

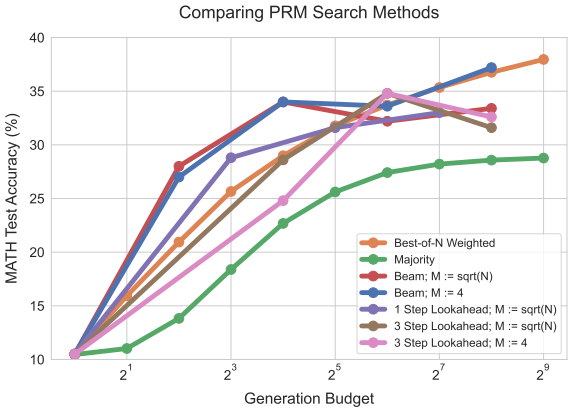


Best-of-N Generate independent complete solutions, then select.
Beam search Score prefixes, keep the top candidates, then expand.
Lookahead Temporarily continue k steps before scoring a prefix.

Lookahead spends additional compute on its temporary continuations.

Beam Search Helps Most at Low Budgets

PaLM 2-S* on MATH: 12,000 training questions and 500 test questions.

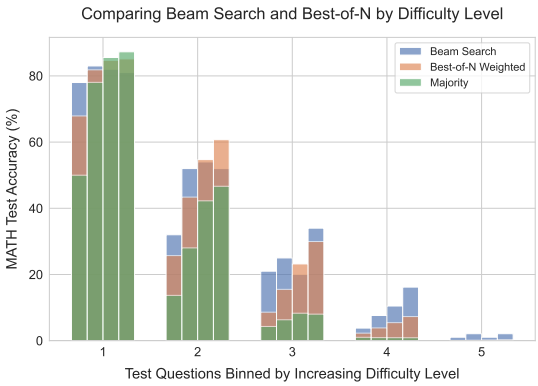


Beam search is strongest at low budgets, but its advantage shrinks as compute increases.

At matched cost, extra lookahead usually gives less benefit.

Difficulty Changes Which Search Strategy Wins

Each difficulty bin has four budgets, left to right: **4, 16, 64, 256**.



Read the top of each color. The bars overlap.

Example, bin 2: beam wins at budget 4.
Best-of-N wins at 256.

Easy questions (bins 1–2)

Beam search can win at small budgets, then stall or decline. Best-of-N catches up.

Medium difficulty (bins 3–4)

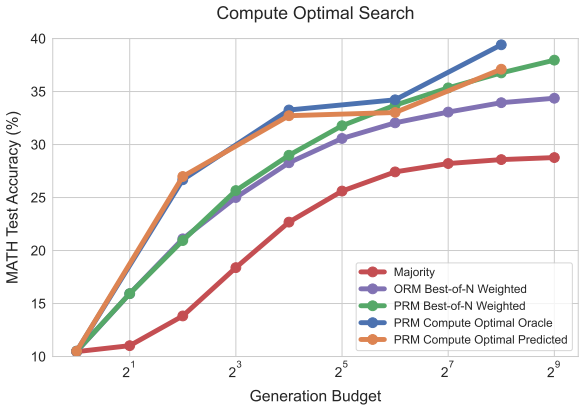
Beam search consistently beats best-of-N at the same budget.

Hardest questions (bin 5)

All three methods make little progress.

Choose the search setting using both difficulty and budget.

Difficulty-Based Allocation Improves Search Efficiency



Difficulty-based allocation improves search efficiency.

At low budgets, 16 generations nearly match best-of-N with 64.

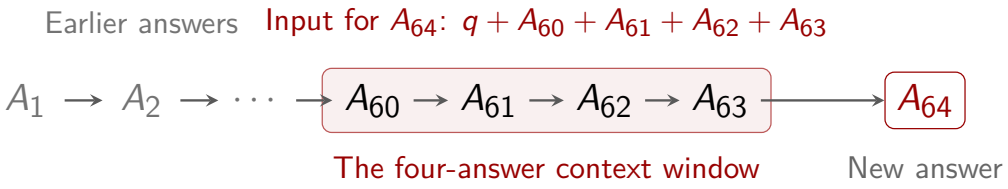
Predicted difficulty loses some of its advantage at higher budgets.

These generation budgets exclude difficulty-estimation cost.

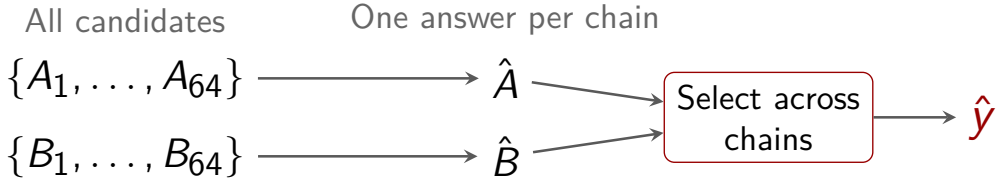
Revising Answers and Selecting the Output

A fine-tuned revision model uses earlier full solutions to write a new answer.

1 Generate Each call sees the question and up to four most recent answers.

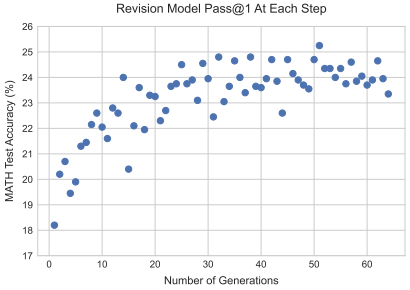


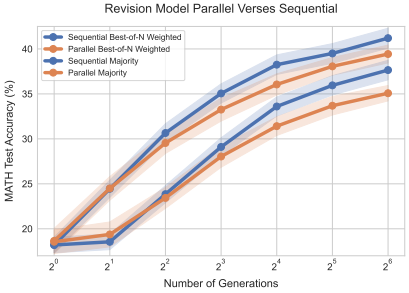
2 Select Keep all candidates, then use a verifier to select an answer.



Training uses incorrect references. At inference, earlier answers may already be correct, and a later revision can undo them.

Revision Quality and Final Answer Accuracy





Left figure: the answer at round t

One question: 4 chains, 64 answers per chain.

Average round-t correctness over the four chains, then over all **500 test questions**.

Most improvement happens in early rounds.

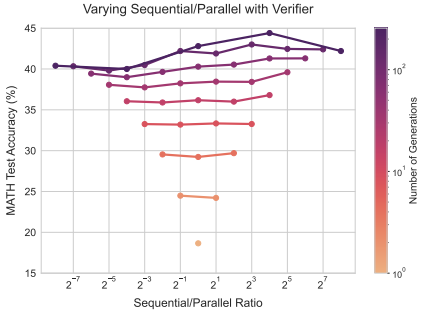
Right figure: one selected answer

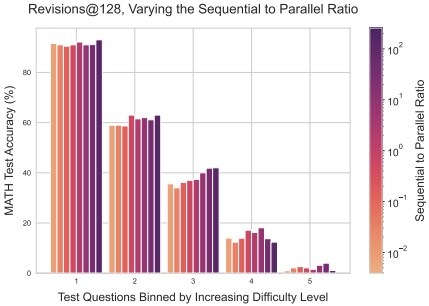
Per question: N independent answers or N sequential answers.

Select one from N, then measure accuracy across all **500 test questions**.

Sequential is slightly better with either selector.

The Best Mix Depends on Difficulty and Budget

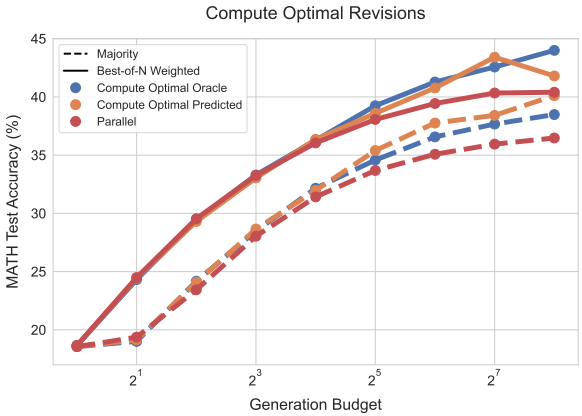




- At high budgets, a mixture can beat either pure parallel or pure sequential generation.
- Easy questions favor long chains; harder questions often benefit from more independent starts.

The best mix depends on both difficulty and budget.

Adaptive Revisions Use the Generation Budget More Efficiently



The reported saving excludes difficulty-estimation cost.

Adaptive revisions keep improving after parallel sampling begins to plateau.

64 candidates can beat parallel best-of-N with 256 candidates.

Both oracle and predicted difficulty improve the allocation.

Pretraining and Inference at Equal Total FLOPs

Extra inference improves the small model. How does it compare with training a larger model?

Hold training tokens fixed and increase parameters by a factor M.

$$X \approx 6PD_{\text{pretrain}}, \qquad Y \approx 2PD_{\text{inference}}$$

P is the parameter count. $D_{\text{inference}}$ is the total inference token workload.

$$\underbrace{X + aY}_{\text{small model, extra inference}} = \underbrace{M(X + Y)}_{\text{large model, greedy decoding}}$$

The larger model costs more both to train and to run. The small model can spend its training savings on inference.

Using R to Compare Total Compute

R is the inference-to-pretraining token ratio used in the cost comparison:

$$R = \frac{D_{\text{inference}}}{D_{\text{pretrain}}}.$$

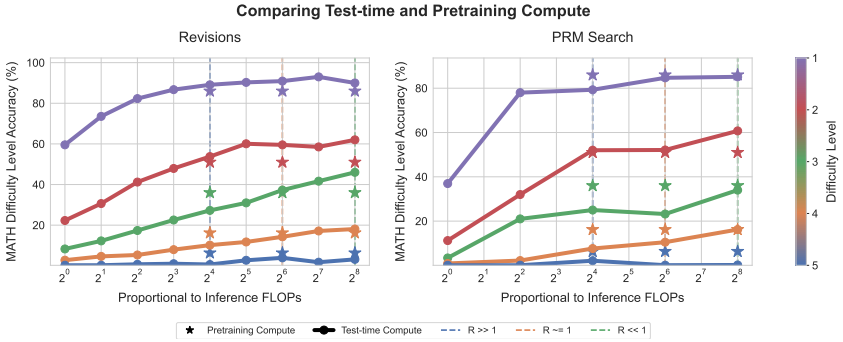
From $X + aY = M(X + Y)$, the small model's inference multiplier is

$$\begin{aligned} a &= M + (M-1)\frac{X}{Y} \\ &= M + (M-1)\frac{6PD_{\text{pretrain}}}{2PD_{\text{inference}}} \\ &= M + 3(M-1)\frac{D_{\text{pretrain}}}{D_{\text{inference}}} = M + \frac{3(M-1)}{R}. \end{aligned}$$

R determines which value of a matches the larger model's total cost.

The accuracy curve measures capability. This equation identifies the equal-cost comparison point.

Extra Inference Can Beat a Larger Model in Some Settings



- Extra inference often wins on easy or medium questions, especially when expected inference use is low.
- A larger model is often better on very hard questions or when expected inference use is high.

The comparison depends on both difficulty and the expected inference workload.

Example: Choosing a Model to Check Answers

We have a batch of written math answers to check before using them as training data.

Compare a larger checker used once with a smaller checker given extra reasoning.

Routine algebra and calculations

The smaller model can understand the solutions.

If extra checking reaches the required accuracy with less total compute, **choose the smaller model**.

Subtle errors in difficult proofs

More reasoning may still leave the smaller model unreliable.

If the larger checker meets the accuracy target within the budget, **choose the larger model**.

Test on answers with known correctness. Count each checker's **pretraining + checking compute**.

Choose the model that meets the accuracy target with less total compute.

Limits of the Comparison

- Estimating difficulty needs substantial compute, excluded from reported budgets.
- Hardest problems gain little from the tested search and revision methods.
- The evidence comes from PaLM 2-S* on MATH, with capability-specific fine-tuning.
- The FLOPs comparison scales parameters at fixed training data and uses oracle difficulty bins.

Conclusions

- The best use of inference compute depends on both the question and the budget.
- Difficulty-based allocation improves PRM search and sequential/parallel revisions, with up to fourfold generation-budget savings in reported comparisons.
- Extra inference can substitute for larger models on some problems and workloads. The hardest problems remain a limitation.

The title's claim is conditional on problem difficulty and expected inference workload.

Thank you

Questions and Discussion

Backup: Constructing Revision Training Data

Build a training pair

- Sample 64 independent answers to one question.
- Choose a correct answer as the target.
- Add 0–4 related incorrect answers as context.
- Prefer close pairs using character edit distance.

An illustrative pair

Question: solve $2x + 3 = 7$.

Incorrect reference

$$2x = 4, \qquad x = 4$$

Correct target

$$2x = 4, \qquad x = 2$$

The target keeps the valid setup and fixes the final calculation.

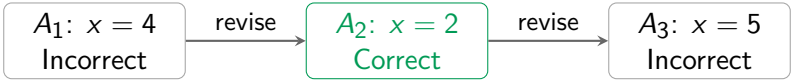
Fine-tune on **question + incorrect references** as input, with the **correct solution** as the target.

Backup: Why the Last Revision Can Be Wrong

Training history Incorrect references, followed by a correct target.

Inference history Generated references can already be correct.

A possible revision chain, illustrated on $2x + 3 = 7$:



About 38% of correct answers became incorrect under the authors' naive revision approach.

Selecting from the history gives the system a chance to recover an earlier correct answer.

Backup: A Complete Beam-Search Example

Suppose each round maintains $N = 6$ candidates and each retained prefix gets $M = 2$ continuations.

1. Generate 6 possible first steps and score their prefixes with the PRM.
2. Keep the top $N/M = 3$ prefixes.
3. Extend each twice, giving $3 \times 2 = 6$ candidates for the next round.
4. Repeat scoring and expansion until completion or 40 rounds.
5. Apply weighted best-of-N to the final candidates and return one answer.

The base model proposes, the PRM scores, search allocates effort, and final selection chooses the output.

Backup: Difficulty Estimation and Policy Selection

Test this idea with **PaLM 2-S* (Codey)** on **MATH**: 12,000 training questions and 500 test questions.

1. Estimate difficulty from 2,048 independent samples per question.
2. Sort questions into five equal-frequency bins, from easy to hard.
3. For each bin and budget, select a setting on one fold and evaluate on the other. Swap folds and average.

Oracle bins Rank by actual correctness rate.
Predicted bins Rank by the verifier's mean final-answer score.

Reported budgets exclude difficulty-estimation cost.

Backup: Difficulty Bins and Cross-Validation

1. Score every question using 2,048 samples. Sort and divide into five quantiles.
2. Within a bin, split questions into folds A and B.
3. Use fold A to select settings at each budget. Evaluate those settings on B.
4. Swap A and B. Average held-out performance.

Oracle and predicted difficulty repeat this procedure with different scoring rules.

Backup: Training Rollouts and Lookahead

PRM training Sample multiple continuations from a prefix to estimate expected final correctness. This supplies supervision.

Lookahead search At inference time, generate up to k extra steps with temperature 0. Use the trained PRM's score at that endpoint to evaluate the current prefix.

The temporary lookahead steps help choose which current prefixes to retain. Beam search is the $k = 0$ case.

Backup: Deriving the Inference Multiplier

$$X + aY = M(X + Y)$$

$$a = M + (M - 1)\frac{X}{Y}$$

$$\frac{X}{Y} = \frac{6PD_{\text{pretrain}}}{2PD_{\text{inference}}} = \frac{3}{R}$$

$$\boxed{a = M + \frac{3(M-1)}{R}}$$

Approximate FLOPs per token: forward $2P$, backward $4P$, training total $6P$.